Four remaining ways to simplify the Lean proofs

DiophantineConsistency after the existing proposal corpus

Prepared for Vladimir Reshetnikov. The four recommendations below concern concrete proof obligations in ProveIt. They survived a review of 23 placed research packets and five incoming archives, with every substantive prior suggestion treated as already available. The contribution is a set of additional source-level simplifications; the underlying mathematics is established.

Main result. Two inductions and several repeated witness manipulations can be removed by choosing a better existing premise: a successor law at zero, the defining membership law of set successor, a linear coefficient estimate used twice, and a common exponential base. Four concrete patches compose cleanly against the pinned sources. Together their eight source files change from 1,089 to 1,045 physical lines, including new helpers and compatibility wrappers.
Evidence level. These are mathematically reviewed, *uncompiled Lean candidates*. Patch applicability, file hashes, preserved source interfaces and finite arithmetic diagnostics were checked. Lean and Lake are unavailable in this session; no kernel acceptance, axiom-set comparison, performance improvement or completed production integration is claimed.

Recommendation	What becomes unnecessary	Candidate lines removed
Specialize multiplication at zero	A genuine internal-omega induction for multiplication by one	13
Read order from set successor	Nonzero-gap/predecessor reconstruction and repeated subtype conversions	16
Compose coefficient envelopes	Expansion of a square into an ordered-pair coefficient sum	13
Normalize powers to one base	Induction on the exponent in a small exponential comparison	2

Source pin: `89226e3921bce3e8949fadd12d3efb6914c7630c`.
Final exclusion cutoff: `7df43cdfacb490cd3ca3acd23e39027efebf8801`.
The eight target files are unchanged between these commits. The later cutoff includes the additional incoming archive that arrived during the review.

Accompanying material: standalone LaTeX and PDF, four narrow patches and their composition, original and proposed versions of the eight files, five Lean examples, novelty and independent-review notes, source metrics, arithmetic diagnostics, and replay scripts.

Contents

1 Scope, evidence, and the meaning of "already applied" 3
1.1 Counterfactual integration versus measured source . . . 3
1.2 The evidence hierarchy . . . 3

2 Specialize the existing multiplication law at zero 4
2.1 The current obligation . . . 4
2.2 The stronger premise is already imported . . . 4
2.3 Why the dependency is legitimate . . . 4
2.4 Increment beyond the prior corpus and expected benefit . . . 5

3 Derive discrete order from set successor membership 5
3.1 Start from the defining semantic law . . . 5
3.2 The old strict-gap theorem becomes a corollary . . . 6
3.3 Concrete consumers and complete accounting . . . 7
3.4 Optional standard successor API . . . 7

4 Apply the linear coefficient estimate twice 8
4.1 View multiplication as an envelope-preserving operation . . . 8
4.2 Place the existing lemma where both clients can use it . . . 8
4.3 Preserve zero budgets, signs, and collisions . . . 9
4.4 Why the earlier coefficient proposals do not subsume this . . . 9
4.5 Optional extension to arbitrary powers . . . 9

5 Reduce the power comparison to a quadratic threshold 10
5.1 The target contains one algebraic parameter . . . 10
5.2 What changes and what remains essential . . . 11

6 Novelty ledger and rejected alternatives 11

7 What was verified and how the patches compose 12
7.1 Exact source accounting . . . 12
7.2 Arithmetic diagnostics . . . 12
7.3 Compiler status . . . 13

8 Recommended integration sequence 13

9 Artifact map and reproducibility 14

Primary source references 14

1 Scope, evidence, and the meaning of "already applied"

The request sets a stronger baseline than the maintained source tree alone. A proposal is excluded if it already appears in the specified research collection or incoming material, even when its code has not landed. Optional sketches, supplementary notes and uncompiled recommendations also count. Renaming such a suggestion, implementing it more fully, or checking it for the first time would not satisfy the request.

The initial inspection used commit `89226e3921bc`. A selective, read-only snapshot retrieved 901 files totaling 10,476,881 bytes, including the maintained Lean subtree, proposal text and the four incoming archives then present. Each retrieved file was checked against its Git blob hash. The final freshness check found commit `7df43cdfacb49` and one additional proposal archive. That archive was downloaded, verified and screened before the recommendation set was finalized.

The final exclusion corpus is therefore 28 packets: a–w under the placed research directory, and five ZIPs under the incoming directory. Principal manuscripts, README summaries, proposal-bearing notes and cumulative exclusion ledgers were inspected, followed by targeted semantic searches for each candidate. The audit does not claim a new proof of global originality or a line-by-line rereading of every historical paper linked from the incoming index.

1.1 Counterfactual integration versus measured source

The existing proposals do not constitute one simultaneously compiled branch. Several are alternative implementations or overlap. Here, "assumed applied" means that their mathematical interfaces and simplifications are already available for deciding novelty. The supplied patches remain concrete patches against a reproducible maintained snapshot. Their line counts measure those patches, not an imagined fully integrated future tree.

This distinction has a practical consequence. If a previously proposed edit has already changed a nearby block, transplant the narrow new argument into that block. Do not overwrite a newer module with a complete candidate file from this archive. The candidate files are review snapshots. In particular, unchanged context around the old Boolean-root proof is not a request to restore that proof after the earlier cancellation simplification.

1.2 The evidence hierarchy

Evidence	What it supports	Status
Pinned source and Git hashes	Which actual declarations and prior proposals were reviewed	Completed
Ordinary mathematical derivations	Validity of the proposed proof routes and their assumptions	Independently reviewed
Patch and interface checks	Concrete applicability, composition, unchanged old headers and exact text deltas	Completed
Finite arithmetic diagnostics	Boundary examples, signs, collisions, zero budgets and exponent threshold	Completed
Lean elaboration and kernel checking	Acceptance of the exact replacement terms and complete modules	Not run
Downstream checks and axiom comparison	Compatibility of the integration and its trust dependencies	Await replay

The mathematical proofs are given below; the finite diagnostics are supplementary. Neither successful patch application nor arithmetic examples are substitutes for Lean checking.

2 Specialize the existing multiplication law at zero

2.1 The current obligation

In ObjectOmegaSemiring, lines 20–35, `mulRel_one_left` establishes

$$\mathrm{MulRel}_H(1, y, y) \qquad \text{for every } y \in \omega^H.$$

Here H is an arbitrary model satisfying the displayed ZF axioms. The relation is the existing definable multiplication graph; it is not multiplication on a separately chosen copy of standard naturals.

The maintained proof chooses a valuation, states the multiplication assertion as an object-language formula, applies internal omega induction, translates the induction hypothesis with the formula's satisfaction theorem, and performs a successor calculation. The induction hypothesis is genuinely used. Consequently, the earlier proposals about removing *vacuous* inductions do not already remove this argument.

2.2 The stronger premise is already imported

The earlier module ObjectMultiplicationLaws, lines 59–62 supplies the relational left-successor law:

$$\mathrm{MulRel}_H(x, y, z) \ \wedge \ \mathrm{AddRel}_H(z, y, w) \quad \Longrightarrow \quad \mathrm{MulRel}_H(\mathrm{succ}(x), y, w).$$

Specialize $x = z = 0$ and $w = y$. Its premises become exactly

$$\mathrm{MulRel}_H(0, y, 0), \qquad \mathrm{AddRel}_H(0, y, y),$$

which are already proved by `mulRel_zero_left` and `addRel_zero_left`. Since the object numeral one is definitionally the successor of the empty set, the conclusion is the required statement.

The complete replacement is:

```
theorem mulRel_one_left (H : ZFAxioms mem) {y : V}
    (hy : mem y (omegaV H)) : MulRel H (natV H 1) y y :=
  mulRel_succ_left H (mulRel_zero_left H hy)
    (addRel_zero_left H hy)
```

The production patch places the final term on one line. Formatting is not the substantive change: the formula, valuation, induction and satisfaction transport in this client all disappear.

2.3 Why the dependency is legitimate

This proof must precede the semiring instance whose `one_mul` field uses it. Appealing to that instance's multiplication identity would create a circular explanation. The proposed term uses only relational laws from the already imported lower layer. The dependency chain goes from `ObjectOmegaSemiring` through

`ObjectDistributivity` to `ObjectMultiplicationLaws` and its addition prerequisites. No import changes are needed.

The argument applies to nonstandard y just as the existing relational theorems do. It specializes a universally quantified model theorem; it does not enumerate the internal naturals with an external recursion. Internal induction remains where the general multiplication laws were established. The redundant client induction is removed.

For independent inspection, the Lean example also gives a second route: apply the right-successor rule to $y \cdot 0 = 0$, simplify $0 + y$, and use already proved commutativity. This alternative is useful for checking the types, but it is not counted as a separate proposal.

2.4 Increment beyond the prior corpus and expected benefit

The nearest earlier proposals replace zero-sum or zero-product proofs whose induction hypotheses are unused by zero/successor decomposition. Here the prior induction is not vacuous. The new observation is that a stronger algebraic rule has already paid for the general induction, and its zero specialization settles the client.

The exact patch removes 13 physical lines and 691 UTF-8 bytes from `ObjectOmegaSemiring.lean`, with no new helper, instance or import. The public theorem statement and all arithmetic definitions are unchanged. This is the lowest-risk first replay because the proposed proof term is a direct application of three existing theorems.

3 Derive discrete order from set successor membership

3.1 Start from the defining semantic law

The internal natural carrier is the subtype

$$N_H = \{x : V \mid x \in \omega^H\}.$$

Its strict order is model membership, and its addition is the existing definable addition graph. The set model already provides

$$u \in \mathrm{succ}(v) \quad \Longleftrightarrow \quad u \in v \ \vee \ u = v. \tag{1}$$

The current strict-gap theorem in ObjectNaturalDiscrete, lines 30–48 instead extracts an additive difference, proves it nonzero, decomposes it as a successor and reconstructs a slack witness. Equation (1) gives a more direct route to the discrete comparison that this proof needs.

First expose the connection between the semantic successor and addition by one:

```
theorem val_add_one (x : Natural H) :
    (x + 1).val = vsucc H x.val := by
  change addValue H x.val (vsucc H (vempty H)) = vsucc H x.val
  rw [addValue_succ_right H x.property (vempty_in_omega H),
    addValue_zero_right H x.property]
```

This lemma changes no operation. It records the two existing recursion equations once, after the semiring instance has been constructed. Its proposed home is `ObjectOmegaSemiring.Natural`.

Proposition 3.1. *For arbitrary $a, b \in N_H$, the existing arithmetic and order satisfy*

$$a < b + 1 \iff a \le b, \qquad a + 1 \le b \iff a < b.$$

Proof. The first equivalence is the following sequence:

$$\begin{aligned} a < b + 1 &\iff a.\mathrm{val} \in (b + 1).\mathrm{val} \\ &\iff a.\mathrm{val} \in \mathrm{succ}(b.\mathrm{val}) \\ &\iff a.\mathrm{val} \in b.\mathrm{val} \ \lor \ a.\mathrm{val} = b.\mathrm{val} \\ &\iff a < b \ \lor \ a = b \\ &\iff a \le b. \end{aligned}$$

The subtype equality step uses extensionality. For the second equivalence, linear-order negation gives

$$a + 1 \le b \iff \neg(b < a + 1) \iff \neg(b \le a) \iff a < b.$$

□

The corresponding main-patch lemmas are short:

```
theorem lt_add_one_iff (a b : N H) : a < b + 1 ↔ a ≤ b := by
  change mem a.val (b + 1).val ↔ a ≤ b
  rw [ObjectOmegaSemiring.Natural.val_add_one H b,
    vsucc_spec H b.val a.val]
  have he : a.val = b.val ↔ a = b :=
    ⟨Subtype.ext, fun h => congrArg Subtype.val h⟩
  exact (or_congr Iff.rfl he).trans le_iff_lt_or_eq.symm

theorem add_one_le_iff (a b : N H) : a + 1 ≤ b ↔ a < b := by
  rw [← not_lt, lt_add_one_iff H b a, not_le]
```

The use of `le_iff_lt_or_eq` is deliberate. The incoming corpus already proposes rebuilding the linear-order instance from its strict part. The new proof does not require the implementation of $\le$ to reduce to a particular disjunction. It uses the semantic strict relation and the general linear-order theorem. If a later refactor makes the strict relation opaque as well, its existing membership bridge should be used at the first line.

3.2 The old strict-gap theorem becomes a corollary

The current canonical-additive-order theorem already says

$$x \le y \quad \iff \quad \exists z \in N_H,\ x + z = y.$$

Apply it to $x + 1 \le y$, then commute the final one past the slack. This gives the old public interface

$$x < y \quad \iff \quad \exists s \in N_H,\ y = x + s + 1$$

without the nonzero-difference and predecessor branches:

```
theorem lt_iff_exists_add_one (x y : N H) :
    x < y ↔ ∃ s : N H, y = x + s + 1 := by
  rw [← add_one_le_iff H x y,
    ObjectNaturalOrder.Natural.le_iff_exists_add H]
  apply exists_congr
  intro s
  simpa only [add_right_comm x 1 s] using
    (eq_comm : (x + s + 1 = y) ↔ (y = x + s + 1))
```

This reverses the dependency of the argument. The defining set successor supplies discrete order first; additive witnesses are then obtained only where the public theorem actually asks for one.

3.3 Concrete consumers and complete accounting

The five-file patch simplifies eight local uses. In `ObjectNaturalDiscrete`, zero/successor decomposition reuses `val_add_one`, and strict-gap existence uses the new comparison bridge. In `ObjectPairingSurjective`, the successor bound, positivity of $a + 1$ and the subtype successor equation become direct applications. `ObjectPairingInjective` supplies its discrete comparison callback without introducing a slack. In `ObjectDivision`, the successor step gets $r + 1 \leq d$ immediately from $r < d$, and the definable induction step reuses the value bridge. See ObjectPairingSurjective, lines 19–84, ObjectPairingInjective, lines 96–102, and ObjectDivision, lines 59–85.

File	Physical-line change
`ObjectOmegaSemiring` (new value bridge)	+5
`ObjectNaturalDiscrete`	−8
`ObjectPairingSurjective`	−6
`ObjectPairingInjective`	−2
`ObjectDivision`	−5
Total, including all three new lemmas	−16

The patch changes no imports or instances. It retains the earlier proposals about multiplication order and Boolean roots as assumed infrastructure; neither is resubmitted here.

3.4 Optional standard successor API

After the local bridges have been checked, a single coherent `SuccAddOrder` instance with successor $x + 1$ can expose Mathlib's standard successor-addition lemmas. The included prototype uses `SuccOrder.ofSuccLeIff` and a local `NoMaxOrder` witness $x < x + 1$. The exact API was inspected at the repository's pinned Mathlib revision.

This optional registration is outside the minimal patch and the 16-line count. It should be adopted only if clients benefit from the standard API enough to justify the instance and import cost. Neither the local bridge nor the optional instance identifies N_H with external $\mathbb{N}$, supplies external well-founded induction, or makes internal intervals externally finite. Those are materially different properties in nonstandard models.

4 Apply the linear coefficient estimate twice

4.1 View multiplication as an envelope-preserving operation

Let

$$E(X) = \sum_i u_i X^{w_i} \in \mathbb{Z}[X], \qquad L = \sum_i |u_i| \geq 0.$$

Suppose $p \in \mathbb{Z}[X]$ satisfies $|\operatorname{coeff}_n(p)| \leq H$ for all n, with $H \geq 0$. The existing theorem `abs_coeff_linear_mul_le` in SplitCoefficientBounds, lines 22–35 proves

$$|\operatorname{coeff}_n(Ep)| \leq HL. \tag{2}$$

The square theorem in SelectiveCoefficientBounds, lines 124–153 currently opens E^2 as a double sum, applies the triangle inequality at both levels, bounds each monomial product, and reconstructs the square of L. Equation (2) already contains everything needed for this second multiplication.

Proposition 4.1. *Under the displayed assumptions,*

$$|\operatorname{coeff}_n(E^2 p)| \leq HL^2 \qquad \textit{for every } n \in \mathbb{N}.$$

Proof. By (2), every coefficient of Ep has absolute value at most HL. Since $HL \geq 0$, apply the same theorem with Ep in place of p and HL in place of H. The resulting bound is $(HL)L = HL^2$. Polynomial multiplication is associative, so $E(Ep) = E^2 p$. □

For completeness, the linear estimate itself follows by summing the bounds

$$|\operatorname{coeff}_n(u_i X^{w_i} p)| \leq |u_i| H.$$

When $w_i > n$, the corresponding shifted coefficient is zero; the existing monomial lemma handles that branch. When several w_i coincide, their contributions are still added. The estimate never replaces a sum by a deduplicated support set, so collisions do not require injectivity assumptions.

4.2 Place the existing lemma where both clients can use it

The import direction is currently

$$\text{SelectiveCoefficientBounds} \longrightarrow \text{SplitCoefficientBounds},$$

where the arrow means that the later module imports the earlier one. The square theorem lives in the earlier module, while the reusable linear theorem lives in the later one. Importing backward would create a cycle.

Move the existing linear proof to `SelectiveCoefficientBounds`, immediately after its monomial bound, under the common name `abs_coeff_encoded_mul_le`. In the later module, preserve the old name `abs_coeff_linear_mul_le` as a wrapper. Then replace the square proof by:

```
have hs : 0 ≤ ∑ i, |u i| :=
  Finset.sum_nonneg fun i _ => abs_nonneg (u i)
have hlinear := abs_coeff_encoded_mul_le w u p H hH hp
have htwice := abs_coeff_encoded_mul_le w u (encoded w u * p)
  (H * ∑ i, |u i|) (mul_nonneg hH hs) hlinear n
simpa only [pow_two, mul_assoc] using htwice
```

The mathematical simplification is repeated application of the same coefficient envelope. Moving the theorem is the dependency adjustment that makes this reuse possible. No new normed-space infrastructure is necessary.

4.3 Preserve zero budgets, signs, and collisions

Three edge cases matter for the proposed abstraction. If $H = 0$, all coefficients of p vanish and the estimate remains valid. If $L = 0$, all input coordinates vanish and the encoded word is zero; no division by L or strict-positivity assumption occurs. If some u_i are negative, the absolute values already account for them. Arbitrary repeated exponents and unmonitored cells are allowed.

These features are part of the current theorem's value. A replacement that silently assumes a Sidon condition, disjoint exponents, positive coordinates, or a nonzero budget would weaken its contract. The supplied replacement retains the exact original hypotheses and conclusion.

The two files together change from 328 to 315 physical lines, including the moved helper and the old-name wrapper. Their UTF-8 size changes from 15,053 to 14,848 bytes. The square declaration itself changes from 29 to 9 physical lines. The complete two-file count is the appropriate metric; counting only the shorter client would hide the helper's cost.

4.4 Why the earlier coefficient proposals do not subsume this

The existing corpus already covers finite coefficient fibers, monomial majorants, row catalogs, sparse placement and generic bounds for literal rows. All of those are assumed available. They establish or simplify a coefficient envelope for a polynomial. The present change concerns how that envelope is propagated through *repeated multiplication* afterward.

The closest inspected material is the finite-majorant and pushforward work in packets c/d/h and the local coefficient-fiber work in t. The newly added 28th packet simplifies a geometric-series bound and explicitly leaves coefficient estimates unchanged. No matching proposal for these two target declarations or this repeated-multiplication proof was found in the scoped audit.

4.5 Optional extension to arbitrary powers

If a higher-power client arises, the same mechanism gives

$$|\operatorname{coeff}_n(E^k p)| \le HL^k \qquad (k \in \mathbb{N})$$

by one induction on k. The base $k = 0$ is the original envelope. The step uses (2) with budget HL^k. It remains valid at $L = 0$, including the conventional $L^0 = 1$ base case.

The archive includes an uncompiled `ConvolutionPowers.lean` candidate. It is intentionally excluded from the minimal migration and its savings. Generality becomes useful when it replaces an actual further client; adding an unused general theorem would not improve the present source count.

5 Reduce the power comparison to a quadratic threshold

5.1 The target contains one algebraic parameter

The theorem `sixteen_pow_bound` in NativePellPower, lines 76–85 states

$$r \geq 2 \quad \Longrightarrow \quad 2^{2r+2} + 2 \leq 16^r. \tag{3}$$

The current proof inducts from two and rewrites both exponential expressions in the successor case. Both powers can instead be expressed in terms of the same quantity

$$t = 4^r \geq 4^2 = 16.$$

The two identities are

$$2^{2r+2} + 2 = 4t + 2, \qquad 16^r = t^2.$$

Thus (3) is the elementary inequality $4t + 2 \leq t^2$ above a fixed threshold.

Proposition 5.1. *For every natural $t \geq 16$, one has $4t + 2 \leq t^2$.*

Proof. Multiplying $16 \leq t$ by $t \geq 0$ gives $16t \leq t^2$. Also $4t + 2 \leq 16t$, because $2 \leq 12t$. Equivalently, over the integers the exact certificate

$$t^2 - 4t - 2 = (t - 16)(t + 12) + 190$$

is nonnegative for $t \geq 16$. □

The proposed Lean replacement stays entirely in natural-number arithmetic:

```
theorem sixteen_pow_bound {r : ℕ} (hr : 2 ≤ r) :
    2 ^ (2 * r + 2) + 2 ≤ 16 ^ r := by
  have ht : 16 ≤ 4 ^ r := by
    simpa using Nat.pow_le_pow_right (by decide : 1 ≤ 4) hr
  calc
    2 ^ (2 * r + 2) + 2 = 4 * 4 ^ r + 2 := by
      rw [pow_add, pow_mul]; ring
    _ ≤ (4 ^ r) ^ 2 := by
      nlinarith [Nat.mul_le_mul_left (4 ^ r) ht]
    _ = 16 ^ r := by
      rw [← pow_mul, mul_comm r 2, pow_mul]; norm_num
```

The multiplied inequality is supplied explicitly to `nlinarith`; the candidate does not rely on the tactic inventing a useful nonlinear product. The power-monotonicity interface was checked in Lean 4.32.0 core. Its positive-base hypothesis accepts the displayed proof of $1 \leq 4$, definitionally the natural-number proposition $0 < 4$.

5.2 What changes and what remains essential

The existing client `two_pow_rank_lt_Y` continues to use the same theorem statement. The replacement removes the induction and the exponent arithmetic in its successor branch. It removes only two physical lines from the complete file, 180 to 178; the gain is the shape of the proof, not a large byte reduction.

The assumption $r \geq 2$ must remain. At $r = 1$, the conclusion would be $18 \leq 16$, which is false. The exact polynomial identity above is interpreted in $\mathbb{Z}$ when it uses subtraction; the actual Lean proof avoids natural-subtraction issues and stays in $\mathbb{N}$.

This differs from the incoming fixed-index Pell-anchor proposal. That proposal targets `large_of_six_le` in `NativePellBootstrap` and replaces a variable-index Pell argument with a checked small-index anchor. Here no Pell sequence or fundamental unit is used. It also differs from the 28th packet's quadratic-margin argument, which derives an interval from a positive expression. The present target is a relation among powers of 2 and 16, reduced to one algebraic variable.

6 Novelty ledger and rejected alternatives

The attached audit groups the excluded material into approximately 45 mechanism families and records the five incoming archives separately. This section gives the distinctions that determine the recommendation set.

Retained proposal	Nearest assumed-available result	Additional step in this report
Multiplication by one	Delete vacuous internal inductions, or centralize formula satisfaction transport	Specialize a previously proved relational successor law to remove an induction that uses its hypothesis
Discrete successor comparison	Generic object-semiring interfaces; rebuild LinearOrder; simplify strict multiplication without discreteness	Read discrete arithmetic from the exact set-successor membership law, then eliminate gap reconstruction
Square coefficient bound	Finite coefficient fibers, row majorants and sparse support bounds	Apply the existing one-factor estimate twice, with its first output envelope as the second input
Power comparison	Small radix facts; fixed-index Pell growth anchor; positive quadratic margin	Normalize both exponential expressions to 4^r and prove a fixed quadratic threshold

Several initially attractive possibilities were discarded. A combined existence-and-uniqueness induction for the internal bound-code relation is already in packet j. Canonical evaluator witnesses occur in earlier notes. Finite-rank library reuse for rails and physical ordering was previously proposed or screened in n/o/s. Certified boundary-to-boundary composition of Wang rows is in packet w's supplementary audit. Sharing the canonical band quotient proof between the existential and marker clients is an exact incoming proposal. The list-head monotonicity cleanup and multiplicative cancellation for Boolean roots are also already present.

These are exclusions, not additional recommendations. The last incoming archive adds periodic-orbit helical proofs, coprime divisibility cancellation, geometric-series bound reuse, a quadratic-margin in-

terval interpretation, signed-prefix division and scalar native-rank comparison. Its manuscript, future questions and substantive candidate code were screened against all four retained mechanisms.

The audit establishes novelty only relative to this specified corpus and cutoff. It makes no claim that specializing recursion laws, composing coefficient estimates, or normalizing exponential bases is new mathematics.

7 What was verified and how the patches compose

7.1 Exact source accounting

The four narrow patches apply sequentially to an isolated copy of the eight source files. The multiplication and successor patches both touch `ObjectOmegaSemiring`, but at disjoint locations. Their composition was checked as one concrete candidate source state.

Module	Original lines	Candidate lines	Reduction
`ObjectOmegaSemiring`	120	112	8
`ObjectNaturalDiscrete`	73	65	8
`ObjectPairingSurjective`	105	99	6
`ObjectPairingInjective`	104	102	2
`ObjectDivision`	179	174	5
`SelectiveCoefficientBounds`	211	206	5
`SplitCoefficientBounds`	117	109	8
`NativePellPower`	180	178	2
Total	1,089	1,045	44

The composed candidate removes 1,636 UTF-8 bytes. Counts include blank lines, comments, helpers and wrappers in the changed source files. Standalone demonstrations, optional extensions, report text and audit scripts are deliverables rather than proposed production code, so they are excluded from this migration metric. The source and candidate hashes are recorded in `evidence/composed-source-validation.json`.

The old theorem headers are preserved in the narrow patches. No executable arithmetic definition, source equation, coordinate convention or numeric constant is changed. No new proof-hole or axiom declaration is introduced in the candidate text. These are source inspections; only a future Lean run can establish the elaborated types and dependency sets of the new proofs.

7.2 Arithmetic diagnostics

The diagnostic script exhaustively checks 78,732 small coefficient-envelope instances. It ranges over three coordinate exponents in $\{0, 1, 2\}$, three signed coordinates in $\{-1, 0, 1\}$, three coefficients of p in that same set, and powers $k = 0, 1, 2, 3$. This includes 2,916 zero-budget instances, 2,916 zero-mass instances and 61,236 instances with colliding coordinate exponents.

All pass. The same script checks $r = 0, \ldots, 100$ in the power statement, finding precisely the expected failures at zero and one, and verifies the quadratic certificate by exact polynomial coefficient arithmetic. These checks address boundary handling in the proposed explanation. The universal convolution proof and the power proof are the mathematical arguments in Sections 4 and 5; the diagnostics do not establish them by sampling.

No standard-natural enumeration is offered as evidence for a theorem about all internal naturals. The two model-theoretic proposals were checked against the actual relational premises and the set-successor law instead.

7.3 Compiler status

The project pins `leanprover/lean4:v4.32.0`. The exact Mathlib revision is `81a5d257c8e410db227a6665ed08f64fea08e997`.
Neither Lean nor Lake is installed in this session. All five Lean examples and all eight modified modules are therefore labeled uncompiled. Prior packets' compiler receipts were not reused as validation of these new edits.

There is no measured claim about elaboration time, heartbeat use, memory, proof-object size, or axiom-set equality. A shorter source proof can have different checking costs. The archive provides replay commands and explicitly separates prototype replay from complete changed-module integration.

8 Recommended integration sequence

1. **Replay the unit specialization first.** Compile the complete `ObjectOmegaSemiring` candidate and its existing checks. Inspect the theorem's inferred type and axioms before using it as the base for the successor migration.
2. **Move the coefficient helper and replace the square proof together.** Build both coefficient modules after the move, retaining the old linear theorem name. The single two-file patch avoids an intermediate backward import or missing public interface.
3. **Replay the elementary power proof independently.** Build `NativePellPower` and its focused checks. Its statement is unchanged, so downstream consumers require no mathematical edits.
4. **Adopt the value and discrete-order bridges as one five-file change.** Build in dependency order, then check the pairing and division endpoints. The minimal patch leaves the instance environment unchanged.
5. **Consider optional generalizations only after the direct changes pass.** Higher-power convolution and `SuccAddOrder` are separate prototypes. Their extra code is not part of the measured reduction.

The supplied `scripts/replay.py` never applies patches. In prototype mode it runs the five standalone examples in a complete checkout. In integration mode it first requires the eight files to match the candidate hashes, builds their actual modules, and runs the selected existing project checks. It stops on failure and records commands, elapsed time and diagnostics. The toolchain check prevents accidentally reporting a replay under a different Lean release as a pinned-environment check.

For a later branch with previous proposals applied, use the mathematical transformations and the individual patches as the review unit. Rebase their narrow changes and regenerate the local line counts. A failed context match is not a reason to replace a newer file wholesale.

9 Artifact map and reproducibility

Path	Purpose
`report.tex`, `report.pdf`	Complete article and compiled reading copy
`patches/01-...04-...`	Four individually reviewable source changes
`patches/combined.patch`	Their checked textual composition
`source-snapshot/`	The eight original files at the source pin
`candidate-source/`	The eight complete files after all four changes
`lean/`	Five uncompiled examples, including the two optional extensions
`audit/`	Prior-proposal ledger, corpus/member hashes and final freshness comparison
`evidence/`	Source metrics, independent mathematical review, diagnostics and toolchain status
`scripts/`	Integrity check, exact source validation, arithmetic diagnostics, Lean replay and report build
`notes/`	More detailed source and dependency notes from the focused reviews

The report builds with XeLaTeX; `scripts/build_report.sh` performs three passes to settle its contents and references. The complete LaTeX source does not depend on downloaded figures or external manuscript fragments. The bundle's SHA-256 manifest covers all final members except the manifest itself.

The portable source validator checks original/candidate hashes, composes the patches in a temporary directory, compares the result with the supplied candidate files and recomputes the text totals. It does not invoke Lean or edit a user checkout. The replay script is the separate route for compiler evidence.

Primary source references

[1] ProveIt, maintained DiophantineConsistency Lean source at the source pin. Individual theorem links appear throughout the article.

[2] ProveIt, placed proof-simplification research, packets a–w, and incoming archives at the final exclusion cutoff. Exact archive names, members and hashes are in the audit manifest.

[3] ProveIt, Lean toolchain and dependency manifest.

[4] Lean 4.32.0, natural-number multiplication and power monotonicity. This verifies the core API used in the power candidate.

[5] Mathlib at `81a5d257`, successor-addition interfaces and successor-order constructors. These concern the optional local-instance prototype, not a new dependency of the main patch.